Practical no 1:

Aim: Write a program for implementing Client Server communication model using TCP

Client server communication model:

The client-server model is a distributed communication framework of network processes among service requestors, clients and service providers. The client-server connection is established through a network or the Internet.

The client-server model is a core network computing concept also building functionality for email exchange and Web/database access. Web technologies and protocols built around the client-server model are:

- Hypertext Transfer Protocol (HTTP)
- Domain Name System (DNS)
- Simple Mail Transfer Protocol (SMTP)
- Telnet

Clients include Web browsers, chat applications, and email software, among others. Servers include Web, database, application, chat and email, etc.

A server manages most processes and stores all data. A client requests specified data or processes. The server relays process output to the client. Clients sometimes handle processing, but require server data resources for completion.

The client-server model differs from a peer-to-peer (P2P) model where communicating systems are the client or server, each with equal status and responsibilities. The P2P model is decentralized networking. The client-server model is centralized networking.

One client-server model drawback is having too many client requests underrun a server and lead to improper functioning or total shutdown. Hackers often use such tactics to terminate specific organizational services through distributed denial-of-service (DDoS) attacks.

TCP:

TCP (Transmission Control Protocol) is a standard that defines how to establish and maintain a network conversation through which application programs can exchange data. TCP works with the Internet Protocol (IP), which defines how computers send packets of data to each other. Together, TCP and IP are the basic rules defining the Internet. The Internet Engineering Task Force (IETF) defines TCP in the Request for Comment (RFC) standards document number 793.

TCP is a connection-oriented protocol, which means a connection is established and maintained until the application programs at each end have finished exchanging messages. It determines how to break application data into packets that networks can deliver, sends packets to and accepts packets from the network layer, manages flow control and -- because it is meant to provide error-free data transmission -- handles retransmission of dropped or

garbled packets as well as acknowledgement of all packets that arrive. In the Open Systems Interconnection (OSI) communication model, TCP covers parts of Layer 4, the transport layer, and parts of Layer 5, the session layer.

Functions/methods:

ServerSocket Class Methods

The java.net.ServerSocket class is used by server applications to obtain a port and listen for client requests.

The ServerSocket class has four constructors

srno.	Method & Description
1	public ServerSocket(int port) throws IOException Attempts to create a server socket bound to the specified port. An exception occurs if the port is already bound by another application.
2	public ServerSocket(int port, int backlog) throws IOException Similar to the previous constructor, the backlog parameter specifies how many incoming clients to store in a wait queue.
3	public ServerSocket(int port, int backlog, InetAddress address) throws IOException Similar to the previous constructor, the InetAddress parameter specifies the local IP address to bind to. The InetAddress is used for servers that may have multiple IP addresses, allowing the server to specify which of its IP addresses to accept client requests on.
4	public ServerSocket() throws IOException Creates an unbound server socket. When using this constructor, use the bind() method when you are ready to bind the server socket.

If the ServerSocket constructor does not throw an exception, it means that your application has successfully bound to the specified port and is ready for client requests.

2.BufferReader funcation:

The Java.io.BufferedReader class reads text from a character-input stream, buffering characters so as to provide for the efficient reading of characters, arrays, and lines.Following are the important points about BufferedReader −

- The buffer size may be specified, or the default size may be used.
- Each read request made of a Reader causes a corresponding read request to be made of the underlying character or byte stream
- Class constructors

Sr.No.	Constructor & Description
1	BufferedReader(Reader in) This creates a buffering character-input stream that uses a default-sized input buffer.
2	BufferedReader(Reader in, intsz) This creates a buffering character-input stream that uses an input buffer of the specified size.

3: inputstreamReader

The Java.io.InputStreamReader class is a bridge from byte streams to character streams.It reads bytes and decodes them into characters using a specified charset.

Class constructors

Sr.No.	Constructor & Description
1	InputStreamReader(InputStream in) This creates an InputStreamReader that uses the default charset.
2	InputStreamReader(InputStream in, Charset cs) This creates an InputStreamReader that uses the given charset.
3	InputStreamReader(InputStream in, CharsetDecoderdec) This creates an InputStreamReader that uses the given charset decoder.
4	InputStreamReader(InputStream in, String charsetName) This creates an InputStreamReader that uses the named charset.

Practical no 2

Aim: Write a program for implementing Client Server communication model using UDP**.**

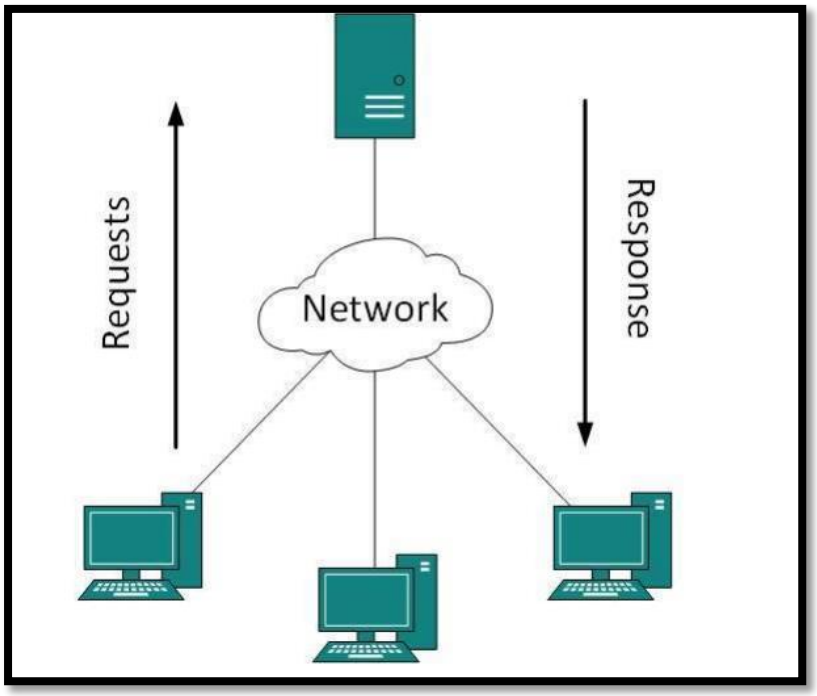
What is client server based program?

Two remote application processes can communicate mainly in two different fashions:

Peer-to-peer: Both remote processes are executing at same level and they exchange data using some shared resource.

Client-Server: One remote process acts as a Client and requests some resource from another application process acting as Server.

In client-server model, any process can act as Server or Client. It is not the type of machine, size of the machine, or its computing power which makes it server; it is the ability of serving request that makes a machine a server.



A system can act as Server and Client simultaneously. That is, one process is acting as Server and another is acting as a client. This may also happen that both client and server processes reside on the same machine.

Communication:

Two processes in client-server model can interact in various ways:

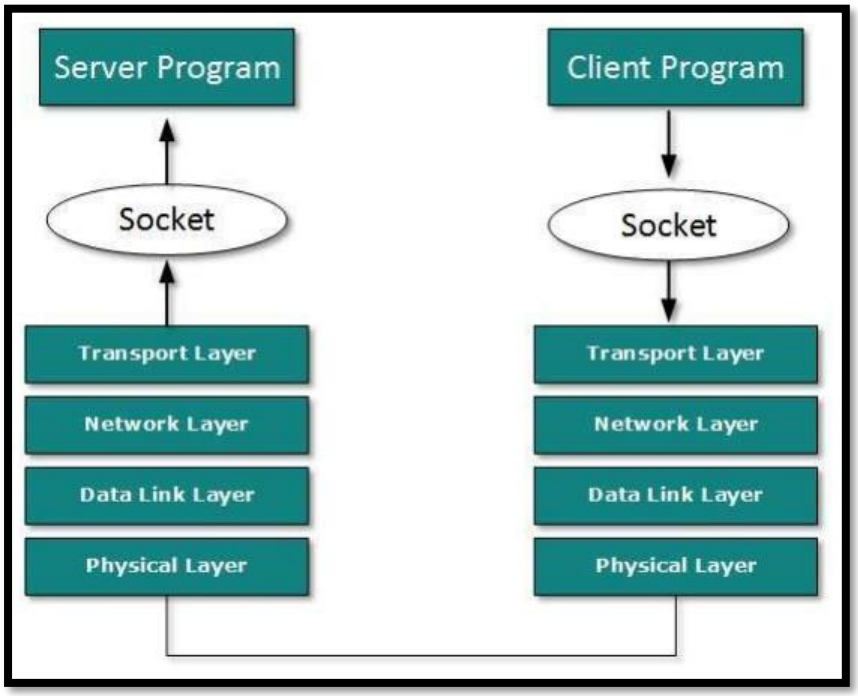
Sockets

Remote Procedure Calls RPC

Sockets :

In this paradigm, the process acting as Server opens a socket using a well-known orknownbyclient port

and waits until some client request comes. The second process acting as a Client also opens a socket but instead of waiting for an incoming request, the client processes ‘requests first’.



When the request is reached to server, it is served. It can either be an information sharing or resource request. Remote Procedure Call This is a mechanism where one process interacts with another by means of procedure calls. One process client calls the procedure lying on remote host. The process on remote host is said to be Server. Both processes are allocated stubs.

This communication happens in the following way:

The client process calls the client stub. It passes all the parameters pertaining to program local to it. All parameters are then packed marshalled and a system call is made to send them to other side of the network.

Kernel sends the data over the network and the other end receives it.

The remote host passes data to the server stub where it is unmarshalled.

parameters are passed to the procedure and the procedure is then executed.

The result is sent back to the client in the same manner.

What is UDP?

The User Datagram Protocol (UDP) is simplest Transport Layer communication protocol available of the TCP/IP protocol suite. It involves minimum amount of communication mechanism. UDP is said to be an unreliable transport protocol but it uses IP services which provides best effort delivery mechanism.

In UDP, the receiver does not generate an acknowledgement of packet received and in turn, the sender does not wait for any acknowledgement of packet sent. This shortcoming makes this protocol unreliable as well as easier on processing.

Requirement of UDP

A question may arise, why do we need an unreliable protocol to transport the data? We deploy UDP where the acknowledgement packets share significant amount of bandwidth along with the actual data. For example, in case of video streaming, thousands of packets are forwarded towards its users. Acknowledging all the packets is troublesome and may contain huge amount of bandwidth wastage. The best delivery mechanism of underlying IP protocol ensures best efforts to deliver its packets, but even if some packets in video streaming get lost, the impact is not calamitous and can be ignored easily. Loss of few packets in video and voice traffic sometimes goes unnoticed.

Features

- UDP is used when acknowledgement of data does not hold any significance.
- UDP is good protocol for data flowing in one direction.
- UDP is simple and suitable for query based communications.
- UDP is not connection oriented.
- UDP does not provide congestion control mechanism.
- UDP does not guarantee ordered delivery of data.
- UDP is stateless.
- UDP is suitable protocol for streaming applications such as VoIP, multimedia streaming.

Functions:

DatagramPacket :

Commonly used Constructors of DatagramPacket class
This constructor is used to receive the packets. DatagramPacket(byte[] barr, int length, InetAddress address, int port): it creates a datagram packet. This constructor is used to send the packets.

RPC serverjava:

public class RpcServer extends java.lang.Object. Class which manages a request queue for a simple RPC-style service. The class is agnostic about the format of RPC arguments / return values.

Practical no 3:

Aim: A multicast Socket example**.**

BroadcastServer.java:

The purpose of a broadcast messenger is to create a server that is responsible for receiving

DatagramPacket , and more. This class represents an Internet address as two fields: hostName
the network. This is called as Broadcasting (to send the packet or message to all the hosts)

Broadcasting is a one-to-all type of communication, i.e. the intention is to send the datagram to all the nodes in the network. Unlike in the case of point-to-point communication, we don't have to know the target host's IP Address. Instead, a broadcast address is used.

InetAddress :
class and provides additional functionality for joining groups. A message sent to the group IP
InetAddress class is Java's encapsulation of an IP address. It is used by most of the other networking classes, including Socket , ServerSocket , URL , DatagramSocket ,

but for receiving the packets addressed to the group, it must join the group. This class
(a String) and address (an int).

MulticastSocket

have to join the group

provides various methods for controlling the flow of multicast packets like setting the ttl, network interface to use etc, along with the major functions of joining and leaving a group.

Constructors :

1. **public MulticastSocket() :** Creates a multicast socket. When using this constructor, we have to explicitly set all the fields such as group address, port number etc.
2. **public MulticastSocket(int port) :** Creates a multicast socket bound on the port specified.
3. **public MulticastSocket(SocketAddress bindaddr) :** Creates a multicast socket and binds it to specified socket address. Creates an unbound socket if address is null.

Practical no 4:

Aim: Write a program to show the object communication using RMI

Remote Method Invocation in Java:

Remote Method Invocation (RMI) is an API which allows an object to invoke a method on an object that exists in another address space, which could be on the same machine or on a remote machine. Through RMI, object running in a JVM present on a computer (Client side) can invoke methods on an object present in another JVM (Server side). RMI creates a public remote server object that enables client and server side communications through simple method calls on the server object.

Working of RMI:

The communication between client and server is handled by using two intermediate obj ects: Stub object (on client side) and Skeleton object (on server side).

Stub Object:

The stub object on the client machine builds an information block and sends this information to the server. The block consists of

- An identifier of the remote object to be used
- Method name which is to be invoked
- Parameters to the remote JVM

Skeleton Object:

The skeleton object passes the request from the stub object to the remote object.
It performs following tasks

- It calls the desired method on the real object present on the server.
  It forwards the parameters received from the stub object to the method.

Practical No. 5

Aim: Show the implementation of web services

What Are Web Services?

Web services are client and server applications that communicate over the World Wide Web's (WWW) Hyper Text Transfer Protocol (HTTP). As described by the World Wide Web Consortium (W3C), web services provide a standard means of interoperating between software applications running on a variety of platforms and frameworks. Web services are characterized by their machine-processable descriptions, thanks to the use of XML. Web services can be combined in a loosely coupled way to achieve complex operations.

Types of Web Services:

On a technical level, web services can be implemented in various ways. The two types of web services can be distinguished as "big" web services and "RESTful" web services.

1) "Big" Web Services:

In Java EE 6, JAX-WS provides the functionality for "big" web services. Big web services use XML messages that follow the Simple Object Access Protocol (SOAP) standard, an XML language defining a message architecture and message formats. Such systems often contain a machine-readable description of the operations offered by the service, written in the Web Services Description Language (WSDL), an XML language for defining interfaces syntactically.

A SOAP-based design must include the following elements.

- A formal contract must be established to describe the interface that the web service offers. WSDL can be used to describe the details of the contract, which may include messages, operations, bindings, and the location of the web service. You may also process SOAP messages in a JAX-WS service without publishing a WSDL.
- The architecture must address complex nonfunctional requirements. Many web service specifications address such requirements and establish a common vocabulary for them. Examples include transactions, security, addressing, trust, coordination, and so on.
- The architecture needs to handle asynchronous processing and invocation. In such cases, the infrastructure provided by standards, such as Web Services Reliable Messaging

(WSRM), and APIs, such as JAX-WS, with their client-side asynchronous invocation support, can be leveraged out of the box.

2) RESTful Web Services:

In Java EE 6, JAX-RS provides the functionality for Representational State Transfer (RESTful) web services. REST is well suited for basic, ad hoc integration scenarios. RESTful web services, often better integrated with HTTP, do not require XML messages or WSDL service–API definitions.

Because RESTful web services use existing well-known W3C and Internet Engineering Task Force (IETF) standards (HTTP, XML, URI, MIME) and have a lightweight infrastructure that allows services to be built with minimal tooling, developing RESTful web services is inexpensive and thus has a very low barrier for adoption.

A RESTful design may be appropriate when the following conditions are met.

- The web services are completely stateless. A good test is to consider whether the interaction can survive a restart of the server.
- A caching infrastructure can be leveraged for performance. If the data that the web service returns is not dynamically generated and can be cached, the caching infrastructure that web servers and other intermediaries inherently provide can be leveraged to improve performance.
- The service producer and service consumer have a mutual understanding of the context and content being passed along. Because there is no formal way to describe the web services interface, both parties must agree out of band on the schemas that describe the data being exchanged and on ways to process it meaningfully.
- RESTful style. Developers can use such technologies as JAX-RS and Asynchronous JavaScript with XML (AJAX) and such toolkits as Direct Web Remoting (DWR) to consume the services in their web applications. Rather than starting from scratch, services can be exposed with XML and consumed by HTML pages without significantly refactoring the existing web site architecture.

Deciding Which Type of Web Service to Use:

JAX-WS: addresses advanced QoS requirements commonly occurring in enterprise computing. When compared to JAX-RS, JAX-WS makes it easier to support the WS-* set of protocols, which provide standards for security and reliability, among other things, and interoperate with other WS-* conforming clients and servers.

JAX-RS: makes it easier to write web applications that apply some or all of the constraints of the REST style to induce desirable properties in the application, such as loose coupling, scalability, and architectural simplicity.You would choose to use JAX-RS for your web application because it is easier for many types of clients to consume RESTful web services while enabling the server side to evolve and scale.

MySQL database:

The data in a MySQL database are stored in tables. A table is a collection of related data, and it consists of columns and rows.

Databases are useful for storing information categorically. A company may have a database with the following tables:

- Employees
- Products
- Customers
- Orders

Practical no 6:

Aim: Implement Xen virtualization and manage with Xen Center

Xen server:

Citirix XenServer is a hypervisor platform that enables the creation and management of virtualized server infrastructure. It is developed by Citrix Systems and is built over the Xen virtual machine hypervisor. XenServer provides server virtualization and monitoring services. It is available in a 64-bit hypervisor platform and can be executed on the entire x86 series of processors.

Cirtix XenServer is among the virtualization solutions provided by Citrix Systems, which consolidates a physical server's computing power into multiple virtual machines, all emulating as a standard server. Citrix XenServer is built to provide the operational requirements of a standard server and supports most server operating systems, such as Linux and Windows Server, on guest server machines.
Through its virtual machine monitoring component, Citrix XenServer manages the allocation and distribution of physical server computing resources among virtual machines and administers their performance and use.

Practical no7:

Aim: Implement virtualization using VMWare ESXi Server and managing with vCenter

Theory:

VMware ESXi :

VMware ESXi (formerly ESX) is an enterprise-class, type-1 hypervisor developed by VMware for deploying and serving virtual computers. As a type-1 hypervisor, ESXi is not a software application that is installed on an operating system (OS); instead, it includes and integrates vital OS components, such as a kernel.

Architecture:

ESX runs on bare metal (without running an operating system) unlike other VMware products. It includes its own kernel: A Linux kernel is started first, and is then used to load a variety of specialized virtualization components, including ESX, which is otherwise known as the vmkernel component. The Linux kernel is the primary virtual machine; it is invoked by the service console. At normal run-time, the vmkernel is running on the bare computer, and the Linux-based service console runs as the first virtual machine. VMware dropped development of ESX at version 4.1, and now uses ESXi, which does not include a Linux kernel. The vmkernel is a microkernel with three interfaces: hardware, guest systems, and the service console (Console OS).

Practical No. 08

Aim : Implement Windows Hyper V virtualization

Theory:

Hyper V:

Hyper-V specifically provides hardware virtualization. That means each virtual machine runs on virtual hardware. Hyper-V lets you create virtual hard drives, virtual switches, and a number of other virtual devices all of which can be added to virtual machines.

Reasons to use virtualization:

Virtualization allows you to:

- Run software that requires an older versions of Windows or non-Windows operating systems.
- Experiment with other operating systems. Hyper-V makes it very easy to create and remove different operating systems.
- Test software on multiple operating systems using multiple virtual machines. With Hyper-V, you can run them all on a single desktop or laptop computer. These virtual machines can be exported and then imported into any other Hyper-V system, including Azure.

Hyper-V Manager Basics

Hyper-V Manager is the most common tool used to interact with Hyper-V. It has these primary features:

- Configure Hyper-V feature settings, such as Live Migration
- Create, destroy, and configure Hyper-V virtual machines
- Create, destroy, and manage virtual hard disks
- Direct control of virtual machine functions; for example: stop, start, and save
- Create, delete, apply, and revert checkpoints
- Configure and control Hyper-V Replication

Hyper-V Virtual Switch:

- Hyper-V Virtual Switch is a software-based layer-2 Ethernet network switch that is available in Hyper-V Manager when you install the Hyper-V server role.
- Hyper-V Virtual Switch includes programmatically managed and extensible capabilities to connect VMs to both virtual networks and the physical network. In addition, Hyper-V Virtual Switch provides policy enforcement for security, isolation, and service levels.
- Hyper-V Virtual Switch includes tenant isolation capabilities, traffic shaping, protection against malicious virtual machines, and simplified troubleshooting.
- With built-in support for Network Device Interface Specification (NDIS) filter drivers and Windows Filtering Platform (WFP) callout drivers, the Hyper-V Virtual Switch enables independent software vendors (ISVs) to create extensible plug-ins, called Virtual Switch Extensions, that can provide enhanced networking and security capabilities. Virtual Switch Extensions that you add to the Hyper-V Virtual Switch are listed in the Virtual Switch Manager feature of Hyper-V Manager.

Practical No:09

Aim: Develop application for Microsoft Azure.

Theory:

Microsoft Azure:

- Microsoft Azure is a cloud computing platform, designed by Microsoft to successfully build, deploy, and manage applications and services through a global network of datacenters.
- Azure can be described as the managed data centers that are used to build, deploy, manage the applications and provide services through a global network. The services provided by Microsoft Azure are PaaS and IaaS. Many programming languages and frameworks are supported by it.

Azure as PaaS (Platform as a Service)

As the name suggests, a platform is provided to clients to develop and deploy software. The clients can focus on the application development rather than having to worry about

hardware and infrastructure. It also takes care of most of the operating systems, servers and networking issues.

Azure as IaaS (Infrastructure as a Service)

It is a managed compute service that gives complete control of the operating systems and the application platform stack to the application developers. It lets the user to access, manage and monitor the data centers by themselves.

Practical No: 10

Aim: Develop application for Google App Engine

Theory:

Google App Engine:

Google App Engine (GAE) is a service for developing and hosting Web applications in Google's data centers, belonging to the platform as a service (PaaS) category of cloud computing. Web applications hosted on GAE are sandboxed and run across multiple servers for redundancy and allowing for scaling of resources according to the traffic requirements of the moment. App Engine automatically allocates additional resources to the servers to accommodate increased load.

Google App Engine is Google's platform as a service offering that allows developers and businesses to build and run applications using Google's advanced infrastructure. These applications are required to be written in one of a few supported languages, namely: Java, Python, PHP and Go. It also requires the use of Google query language and that the database used is Google Big Table. Applications must abide by these standards, so applications either must be developed with GAE in mind or else modified to meet the requirements.

GAE is a platform, so it provides all of the required elements to run and host Web applications, be it on mobile or Web. Without this all-in feature, developers would have to